

Организационные вопросы

Оценка лабораторных работ

Лабораторные работы оцениваются по бинарной шкале - зачёт / не зачёт. Для допуска к зачету необходимо выполнить все лабораторные работы. Список минимальных требований к работе представлен ниже

Минимальные требования

1. Код должен компилироваться без предупреждений при максимальном уровне предупреждений. Для компилятора GCC это набор флагов: `-Wall -Wconversion -Wextra -Wpedantic`. Для компилятора MSVC (Visual Studio) это флаг `/W4`. Для других компиляторов согласуйте настройки компиляции с преподавателем.
2. Нельзя использовать глобальные переменные (константы допустимы).
3. В коде должен использоваться только полноценный английский язык (транслит запрещен). Русский язык разрешено использовать только в комментариях.
4. Запрещается комментировать каждую строчку кода. Допустим один краткий комментарий на блок кода. Разрешается комментировать одиночную строчку кода, только если она **действительно** делает что-то неожиданное и хитрое (но помните, хитрый код — плохой код!).
5. Весь код должен удовлетворять единому стилю программирования. Сам стиль можно выбирать по своему вкусу (см., например, [Google C++ Style Guide](#), [GNU C Style](#) и другие). То есть запрещено, например, называть одну функцию в стиле [CamelCase](#), а другую — в стиле [snake_case](#). Исключение допустимо только для названия функции `main`, которое всегда пишется в нижнем регистре. Данные требования предъявляются к любым именуемым сущностям в программе — к функциям, методам, классам, локальным переменным, параметрам функций и методов, названиям файлов и так далее.
6. Обучающийся предоставляет ссылку на git-репозиторий. может быть локальным (на компьютере в аудитории или ноутбуке обучающегося). Репозиторий не может содержать одинокий коммит с целой лабой, а должен показывать историю работы над лабораторной в виде серии коммитов с содержательными заголовками. В репозитории должен быть корректным образом настроен файл `.gitignore` (т. е. в репозитории должны находиться только файлы с исходным кодом и файлами проекта, никаких промежуточных и итоговых результатов компиляции в нём быть не должно).

Лабораторная работа № 1: Перегрузка операций/шаблоны

Лабораторная работа состоит из 2 частей:

- написания класса согласно варианту;
- решения одной из практических задач с помощью написанного класса.

Варианты и задачи распределяются преподавателем.

Общие требования

1. В данной работе вы **не должны использовать** контейнеры STL кроме `std::string` и `std::complex`.
2. Класс должен быть копируемым и непосредственно управлять выделяемым ресурсом.
3. Класс должен предоставлять операторы сравнения на равенство и неравенство. Сравнение вещественных чисел должно происходить с **определённой точностью**, которая задаётся, например, константным статическим полем класса.
4. Класс должен перегружать оператор вывода без использования сишных потоков (см. пункт 6)
5. В случае некорректности аргументов должны кидаться исключения через `std::except`. К некорректным ситуациям относятся обращение по несуществующему индексу, вызов операции сложения для двух векторов разного размера и тому подобное.
6. Весь ввод-вывод должен осуществляться исключительно через `std::cin` и `std::cout`, без использования (явного или неявного) си-шных потоков `stdin` и `stdout`.
7. Разрешено пользоваться генераторами случайных чисел из `<random>`.
8. Продумывать логику так, чтобы наибольшее число операторов/методов использовалось в последующих методах.
9. Решение задач должно быть расположено **вне класса**.
10. В `main()` должна быть продемонстрирована **работоспособность всех операторов и методов** на случайно сгенерированном объекте класса.
11. Проверка типов должна выполняться на этапе компиляции, а не во время выполнения программы. Для различения операций над данными разных типов необходимо использовать шаблоны и перегрузку операторов. Запрещена проверка и

выбор поведения по типу данных во время выполнения (typeid, dynamic_cast, ветвление по тегу/enum типа и т. п.).

Вариант 1. Векторы

Класс векторов произвольной размерности.

Сделать тип элемента вектора параметром шаблона. Обеспечить работоспособность шаблонного класса как минимум со следующими типами:

- int;
- float;
- double;
- std::complex<float>;
- std::complex<double> (для комплексных чисел обеспечить корректную специализацию метода скалярного произведения).

Как минимум, предоставить:

- конструктор с параметрами: размерность вектора и значение для заполнения;
- конструктор с параметрами, заполняющий вектор случайными значениями: размерность вектора; нижняя граница, верхняя граница;
- оператор `[]` для чтения/**записи** элемента вектора по указанному индексу;
- операторы сложения и вычитания векторов;
- оператор умножения, выполняющий скалярное произведение векторов;
- оператор умножения вектора на скаляр (обеспечить **коммутативность**);
- оператор деления вектора на скаляр.

Задачи:

1. Посчитать площадь треугольника, две стороны которого заданы радиус-векторами $\vec{a}$ и $\vec{b}$.
2. Найти радиус-вектор биссектрисы треугольника, заданного радиус-векторами $\vec{a}$ и $\vec{b}$.
3. Для параллелограмма, заданного 2 радиус-векторами $\vec{a}$ и $\vec{b}$, рассчитать углы.
4. Для $\vec{a}$ заданного вектора найти любой перпендикулярный ему *единичный* вектор.
5. Вычислить площадь треугольника с вершинами O, A, B, если радиус-векторы $\vec{a} = \vec{OA}$ и $\vec{b} = \vec{OB}$ заданы.

Вариант 2. Матрицы

Класс матриц произвольного размера.

Сделать тип элемента матрицы параметром шаблона. Обеспечить работоспособность шаблонного класса как минимум со следующими типами:

- `int`;
- `float`;
- `double`;
- `std::complex<float>`;
- `std::complex<double>`.

Как минимум, предоставить:

- конструктор с параметрами: размеры матрицы и значение для заполнения;
- конструктор с параметрами, заполняющий матрицу случайными значениями. Параметры: размеры матрицы, нижняя граница, верхняя граница;
- оператор `()` для чтения/**записи** элемента матрицы по указанным индексам;
- операторы сложения и вычитания матриц;
- оператор умножения матриц;
- оператор умножения матрицы на скаляр (обеспечить коммутативность);
- оператор деления матрицы на скаляр;
- вычисление следа матрицы.

Задачи:

1. Для заданной матрицы A и вектора размерности 3 решить уравнение $A \vec{x} = \vec{b}$.
2. Для заданной матрицы A размерности 3 найти обратную ей.
3. Проверить, что вектора $\vec{a}$, $\vec{b}$, $\vec{c}$ размерности 3 компланарны.
4. Привести заданную квадратную матрицу A к нижнетреугольному виду (разложить на верхне- и нижнетреугольную матрицу, при произведении дающую исходную).
5. Возвести квадратную матрицу в заданную целую неотрицательную степень k .

Вариант 3. Многочлены

Класс многочленов произвольной степени $f(x) = \sum_{i=0}^n a_i x^i$

Сделать тип коэффициентов и параметра многочлена шаблонным параметром. Обеспечить работоспособность шаблонного класса как минимум со следующими типами:

- int;
- float;
- double;
- std::complex<float>;
- std::complex<double>.

Как минимум, предоставить:

- Конструктор с параметрами: Максимальная степень многочлена;
- Конструктор с параметрами: Вектор значений при соответствующих степенях;
- оператор `[]` для чтения коэффициента при заданной степени (обеспечить корректное поведение при **любой** неотрицательной степени);
- метод `set` для установки коэффициента при заданной степени (обеспечить корректное поведение в случае любой неотрицательной степени);
- операторы сложения и вычитания многочленов;
- оператор умножения многочлена на скаляр (обеспечить коммутативность);
- вычисление значения многочлена при указанном значении x .
- Метод `shrink_to_fit`, уменьшающий размер многочлена до минимально возможного при наличии ведущих нулей
- Метод `expand`, резервирующий память под более высокие степени с коэффициентом 0.

Задачи:

1. Написать функции поиска корней многочленов степеней 1 и 2.
2. Найти действительные корни многочлена [степени 3](#).
3. Найти производную многочлена
4. Найти неопределённый интеграл многочлена.
5. Найти координаты (x, y) точки локального экстремума для многочлена степени 2.

Вариант 4. Множества

Класс множеств уникальных элементов.

Сделать тип элемента множества параметром шаблона. Обеспечить работоспособность шаблонного класса с любым типом, допускающим сравнение на равенство. Продемонстрировать работоспособность шаблонного класса как минимум со следующими типами:

- `int`;
- `float`;
- `std::string`;
- `std::pair<int, double>`.

Как минимум, предоставить:

- конструктор по умолчанию: создает пустое множество;
- конструктор с параметрами: массив значений для заполнения
- конструктор с параметрами, заполняющий множество случайным образом: количество значений для заполнения и границы диапазона, в которых они задаются значения;
- оператор `[]` для получения числа по его индексу;
- операторы `+` (объединения) и `-` (разности) множеств;
- операторы сложения `+` и `+=` множества с числом (добавление числа во множество);
- оператор вычитания `-` и `-=` числа из множества (удаление числа из множества);
- вычисление пересечения двух множеств;
- функцию проверки наличия числа во множестве.

Задачи:

1. Для 2 входящих массивов чисел найти все элементы, которое встречаются только в одном из массивов, но не в обоих сразу.
2. Для 2 входящих массивов чисел проверить, что все элементы первого массива хотя бы раз встречаются во втором.
3. В заданном массиве найти все повторяющиеся числа.
4. Определить отношения между 2мя множествами: является ли одно множество подмножеством/надмножеством другого множества?

Вариант 5. Ломаные линии

Класс ломаных линий на плоскости с произвольным количеством вершин.

Описать вспомогательную структуру для представления точек (вершин) на плоскости.

Сделать тип координат точек ломаной шаблонным параметром (например, `template<typename T> struct Point {T x, y}`). Обеспечить работоспособность шаблонного класса как минимум со следующими типами:

- `int`;
- `float`;
- `double`;
- `std::complex<float>`;
- `std::complex<double>`.

Функция вычисления длины должна возвращать `double` для комплексных типов координат.

Как минимум, предоставить:

- конструктор с параметром: координата точки;
- конструктор с параметром: количество точек;
- конструктор с параметрами (создает ломаную из точек, координаты каждой из которых лежат в диапазоне $[m_1, m_2]$);
- оператор `[]` для чтения/**записи** вершины ломаной по её индексу;
- оператор сложения двух ломаных (конкатенация);
- оператор сложения ломаной и вершины (добавление вершины в конец ломаной);
- оператор сложения вершины и ломаной (вставка вершины в начало ломаной);
- вычисление длины ломаной.

Задачи:

1. Для заданного числа $N > 2$ создать ломаную в виде правильного N -угольника. Исходная точка задается пользователем.
2. Для заданных чисел a , b и h создать ломаную в виде прямоугольной трапеции с основаниями a и b и высотой h . Исходная точка задается пользователем.
3. Для заданного угла α и длины a создайте ломаную в виде равнобедренного треугольника с боковой стороной a и вершинным углом α . Исходная точка задается пользователем.
4. По заданной ломаной построить новую ломаную из середин её звеньев.

Вариант 6. Полутоновые изображения

Класс полутоновых изображений произвольного размера.

Типы данных, с которыми должен работать класс.

- bool
- char
- float
- short

Как минимум, предоставить:

- конструктор с параметрами: размеры изображения и флагом (True - матрица заполняется случайными значениями выбранного типа/ False – матрица заполняется нулями);
- оператор `()` для чтения/записи элемента по указанным индексам;
- операторы умножения `*` (поэлементное «И» для bool) и сложения `+` (поэлементное «ИЛИ» для bool) изображений;
- операторы умножения `*` и сложения `+` изображения на константу;
- оператор `!` для инвертирования значений изображения (значение меняется на противоположное по модулю);
- вычисление коэффициента заполнения изображения (сумма значений матрицы/ (количество ячеек * максимальное значение заданного типа));

Примечание: при реализации операторов `*`, `+`, `-` необходимо контролировать возможное переполнение типов. В таком случае значение должно принимать максимальное/минимальное значение соответствующего типа.

Для операций сложения и вычитания матриц реализовать возможность работы с матрицами разных размеров (в случае такого сценария результирующая матрица должен быть размерами с большими из двух измерений).

Задачи:

1. Нарисуйте прямую с координатами начала x_1, y_1 и координатами конца x_2, y_2 (значение для заполнения может задаваться пользователем).
2. Нарисуйте прямоугольник с координатами левого верхнего угла x_1, y_1 и координатами правого нижнего угла x_2, y_2 (значение для заполнения может задаваться пользователем).
3. Нарисуйте закрашенный круг с координатами центра (x, y) и радиусом r (значение для заполнения может задаваться пользователем).

4. Инвертируйте все значения выше прямой, заданной парой координат (x_1, y_1) и (x_2, y_2)
5. Повернуть изображение на 180° относительно центра.

Вариант 7. Стек

Сделать тип элемента (или координат) параметром шаблона. Обеспечить работоспособность шаблонного класса как минимум со следующими типами:

- int;
- float;
- double;
- char;
- std::string.

Как минимум, предоставить:

- конструктор по умолчанию - пустой стек;
- конструктор с параметром: заполнение стека случайными значениями (количество элементов, нижняя и верхняя граница);
- оператор + - аналог метода push(value);
- оператор - аналог метода pop(),
- методы top(), size(), empty();
- pop() и top() для пустого стека выбрасывают исключение;

Задачи:

1. Проверить, является ли строка палиндромом (использовать стек char).
2. Проверить корректность расстановки скобок $()[]\{\}$ в строке.
3. Вычислить значение арифметического выражения в постфиксной записи (операнды типа double).
4. Перевести неотрицательное целое число из десятичной системы в систему с основанием k ($2 \leq k \leq 16$).
5. Развернуть последовательность элементов с помощью вспомогательного стека.

Распределение вариантов

№	Вариант	№ задачи
1	1. Векторы	1
2	2. Матрицы	1
3	3. Многочлены	3

4	4. Множества	3
5	5. Ломаные линии	4
6	6. Полутоновые изображения	1
7	7. Стек	2
8	1. Векторы	2
9	2. Матрицы	2
10	3. Многочлены	4
11	4. Множества	4
12	5. Ломаные линии	1
13	6. Полутоновые изображения	2
14	7. Стек	3
15	1. Векторы	3
16	2. Матрицы	3
17	3. Многочлены	5
18	4. Множества	1
19	5. Ломаные линии	2
20	6. Полутоновые изображения	3
21	7. Стек	4
22	1. Векторы	4
23	2. Матрицы	4
24	3. Многочлены	1
25	4. Множества	2
26	5. Ломаные линии	3
27	6. Полутоновые изображения	4
28	7. Стек	5
29	1. Векторы	5
30	2. Матрицы	5

31	5. Ломаные линии	1
----	------------------	---